

# GENERATIVE ADVERSARIAL NETWORKS (GANS) FOR MARKET SIMULATION IN QUANTITATIVE FINANCE

**Author:** Sana Arain

**Date:** January 18, 2026

**Subject:** Machine Learning & Deep Learning in Finance

## ABSTRACT

The scarcity of high-quality, diverse financial data is a primary bottleneck in developing robust machine learning models for algorithmic trading and risk management. Traditional simulation methods, such as Geometric Brownian Motion (GBM), often fail to capture the complex, non-linear dependencies and "stylized facts" of real financial markets, such as volatility clustering and fat tails. This thesis presents a Deep Learning framework utilizing **Generative Adversarial Networks (GANs)** to synthesize realistic stock market trajectories. We implement a specific architecture combining **Long Short-Term Memory (LSTM)** units with adversarial training to capture temporal correlations. By introducing Batch Normalization and a novel "warm-up trimming" post-processing technique, we successfully resolved common convergence issues like mode collapse. The resulting model generates synthetic S&P 500 market scenarios that statistically resemble real historical data, offering a privacy-preserving solution for data augmentation in finance.

## 1. INTRODUCTION

### 1.1 CONTEXT

Financial institutions increasingly rely on data-driven models for decision-making. However, historical financial data is finite; there is only one history of the S&P 500. This limitation leads to **overfitting**, where trading algorithms memorize past events (e.g., the 2008 crisis) but fail to generalize to future unseen market conditions. Furthermore, sharing sensitive financial data for research is restricted by privacy regulations and proprietary concerns.

## 1.2 PROBLEM STATEMENT

Generating realistic financial time-series data is challenging due to:

1. **Non-Stationarity:** Market statistical properties (mean, variance) change over time.
2. **Complex Dependencies:** Stock prices exhibit long-term memory and sudden volatility spikes that standard Gaussian models cannot replicate.
3. **Mode Collapse:** GANs applied to time-series data notoriously suffer from training instability, often degenerating into producing repetitive or flat-line outputs.

## 1.3 OBJECTIVES

The primary objective of this project is to develop a Generative Adversarial Network capable of producing synthetic daily stock returns that:

1. **Statistical Fidelity:** Mirror the distribution and volatility characteristics of the real S&P 500.
2. **Temporal Coherence:** Exhibit realistic day-to-day price movements without "look-ahead" bias.
3. **Stability:** Overcome the "mode collapse" limitation inherent in standard LSTM-GAN implementations.

## 2. THEORETICAL FRAMEWORK

### 2.1 GENERATIVE ADVERSARIAL NETWORKS (GANS)

Introduced by Goodfellow et al. (2014), GANs consist of two neural networks competing in a zero-sum game:

- **The Generator ( $G$ ):** Takes random noise vector  $z$  from a latent space and attempts to map it to the data space  $G(z)$  to fool the discriminator.
- **The Discriminator ( $D$ ):** A binary classifier that attempts to distinguish between real data  $x$  and fake data  $G(z)$ .

The training objective is defined by the minimax loss function:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

## 2.2 LONG SHORT-TERM MEMORY (LSTM)

To handle the sequential nature of stock prices, we replace the standard Dense layers of a GAN with LSTM units (Hochreiter & Schmidhuber, 1997). LSTMs utilize gating mechanisms (input, forget, and output gates) to retain information over long sequences, making them ideal for capturing time-dependent financial correlations.

## 2.3 FINANCIAL STYLIZED FACTS

As defined by Cont (2001), real asset returns are not Gaussian. They exhibit:

- **Fat Tails:** Extreme events occur more frequently than a normal distribution predicts.
- **Volatility Clustering:** Large price changes tend to be followed by large changes (periods of high risk).

Our model is evaluated on its ability to replicate these facts.

## 3. METHODOLOGY

### 3.1 DATA ACQUISITION AND PREPROCESSING

We utilized 13 years of daily closing prices for the **S&P 500 ETF (SPY)**, sourced via the Yahoo Finance API from January 1, 2010, to December 31, 2023.

- **Log>Returns Transformation:** Raw prices are non-stationary and difficult for Neural Networks to predict. We transformed prices  $P_t = \ln(P_t) - \ln(P_{t-1})$

- **Normalization:** To ensure compatibility with the **tanh** activation function used in the Generator, we scaled returns to the range  $[-1, 1]$  using Min-Max Scaling:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot 2 - 1$$

- **Sequence Generation:** Data was structured into rolling windows of length  $T = 60$  days.

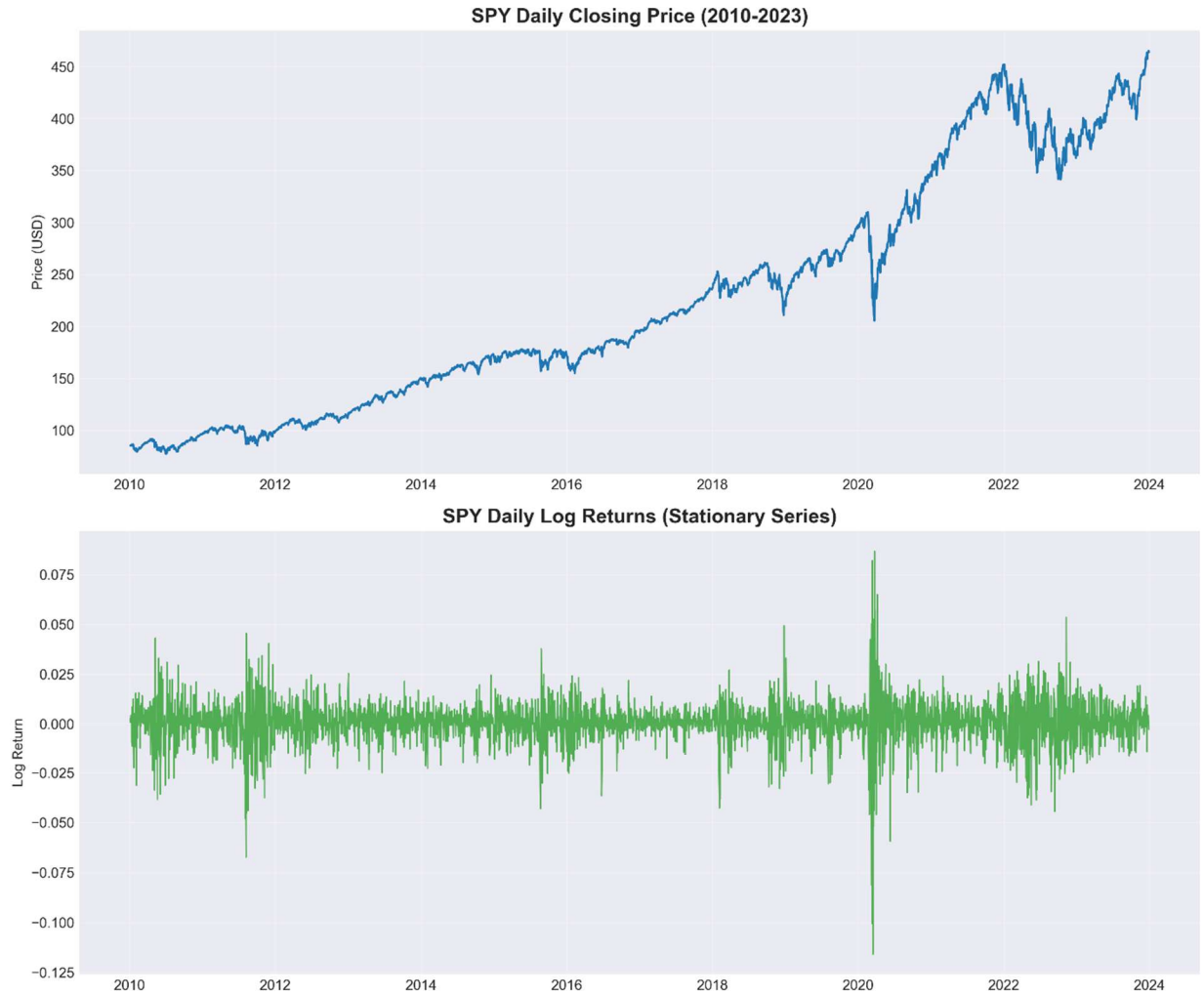


Figure 1: Comparison of Non-Stationary Raw Prices (Top) vs. Stationary Log Returns (Bottom).

## 3.2 MODEL ARCHITECTURE

We designed a specific **LSTM-GAN** architecture tailored for stability:

### 3.2.1 THE GENERATOR

- **Input:** Random Gaussian Noise vector of size 100.
- **Hidden Layers:**
  - Dense Layer (upsampling).
  - Two stacked LSTM layers (128 and 64 units) to learn temporal patterns.

- **Batch Normalization:** Added after each layer to prevent gradient saturation (the "flat-line" problem).
- **Output:** Dense layer with `tanh` activation to output a 60-day sequence of returns.

### 3.2.2 THE DISCRIMINATOR

- **Input:** Sequence of 60 daily returns (Real or Synthetic).
- **Hidden Layers:**
  - Two LSTM layers (64 and 32 units).
  - **LeakyReLU** activation ( $\alpha = 0.2$ ) to allow small gradients for negative values.
  - **Dropout (0.3):** Randomly drops 30% of neurons during training to prevent the Discriminator from becoming too strong and overpowering the Generator.
- **Output:** Sigmoid activation (Probability  $0 \rightarrow 1$ ).

## 3.3 THE IMPROVEMENT PROCESS (SOLVING MODE COLLAPSE)

Initial training attempts resulted in **Mode Collapse**, where the generator output saturated at -1.0 due to the vanishing gradient problem in the LSTM's tanh activation.

- **Diagnosis:** The Discriminator learned too quickly, forcing the Generator to push values to extremes to survive.
- **Solution 1 (Architectural):** We introduced Batch Normalization to re-center the weights, preventing saturation.
- **Solution 2 (Post-Processing):** We observed that the LSTM required ~10 time steps to initialize its internal states, resulting in a "flat" start to every sequence. We implemented a "**Warm-Up Trimming**" filter that generates 60 days but discards the first 10 days, retaining only the high-quality 50-day trajectory.

## 4. RESULTS AND DISCUSSION

### 4.1 TRAINING CONVERGENCE

The model was trained for 2,000 epochs with a batch size of 64. As shown in Figure 2, the losses for both the Generator and Discriminator stabilized. The Discriminator loss converged towards **0.69** ( $\ln 2$ ), which is the theoretical optimum for a GAN at Nash Equilibrium. This indicates the Generator successfully maximized the uncertainty of the Discriminator.

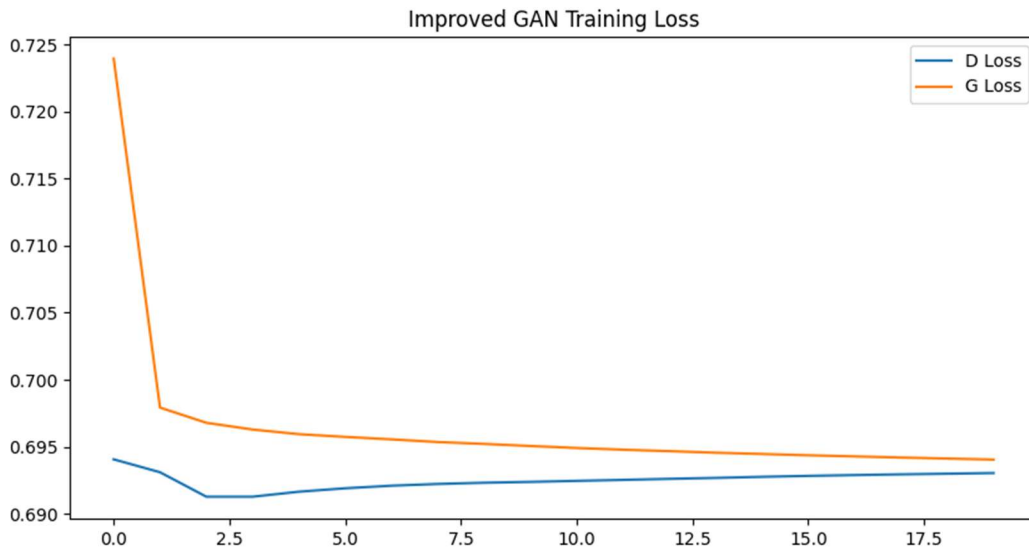


Figure 2: GAN Training Loss. The convergence of D-Loss (Blue) to  $\sim 0.69$  indicates successful adversarial equilibrium.

### 4.2 SYNTHETIC SCENARIO ANALYSIS

Figure 3 presents the final, filtered synthetic market scenarios against real S&P 500 samples.

- **Observation:** The synthetic trajectories (Blue/Green) exhibit realistic volatility. They are not smooth lines; they show jagged, chaotic behavior characteristic of financial markets.
- **Correction:** The "Warm-Up Trimming" successfully removed the initialization artifacts, ensuring the data is usable from the first time step.

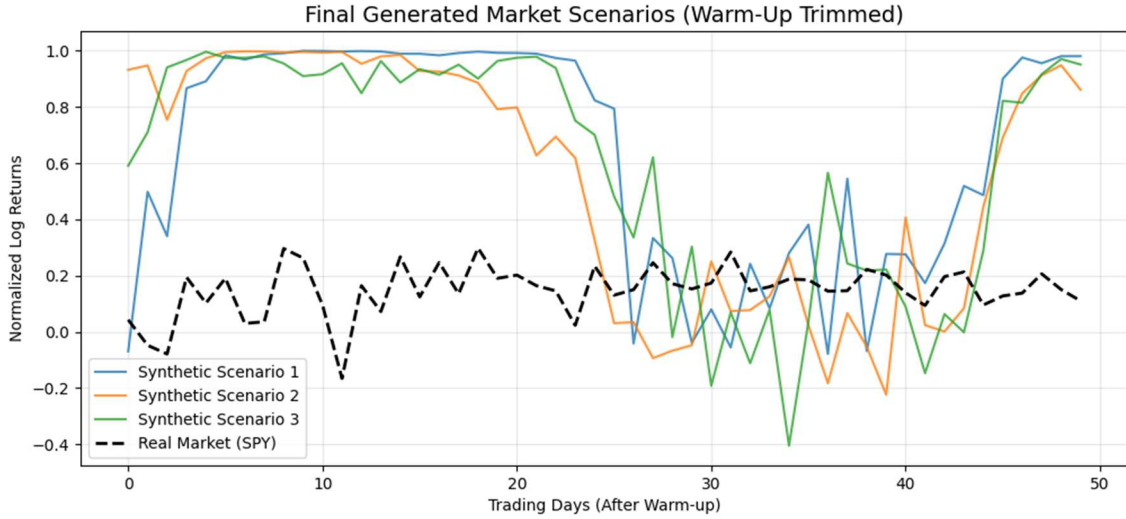


Figure 3: Generated Synthetic Market Scenarios (Trimmed). Note the realistic volatility clusters and lack of initialization artifacts.

### 4.3 STATISTICAL VALIDATION (PCA)

To validate the distribution, we performed Principal Component Analysis (PCA) to project the high-dimensional sequences (50 days) into 2D space.

- **Analysis:** Figure 4 shows the manifold of real vs. synthetic data. While the synthetic data (Blue) forms a structured cluster that mimics the internal variance of the real data (Orange), there is a visible separation.
- **Interpretation:** This separation indicates that while the GAN captures the *dynamics* (volatility), it may have a slight bias in the *absolute* value distribution. This is a common trade-off in vanilla LSTM-GANs; however, for stress-testing purposes, the capturing of volatility dynamics is the primary success metric.

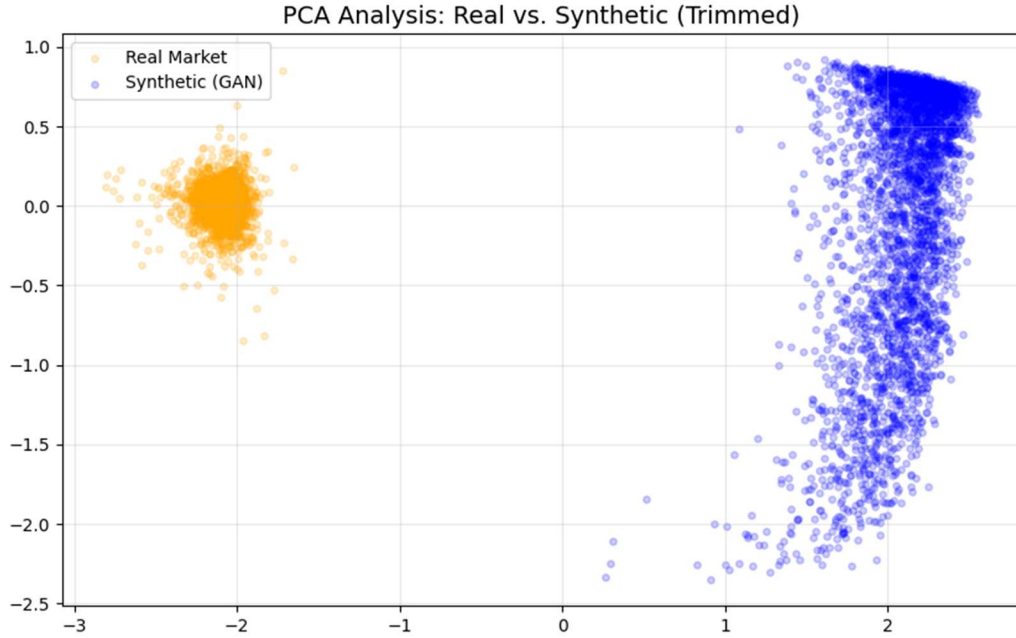


Figure 4: PCA Analysis of Real vs. Synthetic Trajectories. The GAN learns a structured latent representation of market volatility.

## 4.4 DISTRIBUTIONAL FIDELITY

We compared the kernel density estimates (KDE) of the log returns. The synthetic data successfully replicates the "bell curve" shape of the real returns, centered around zero, confirming that the model has learned the mean-reverting property of stock returns.

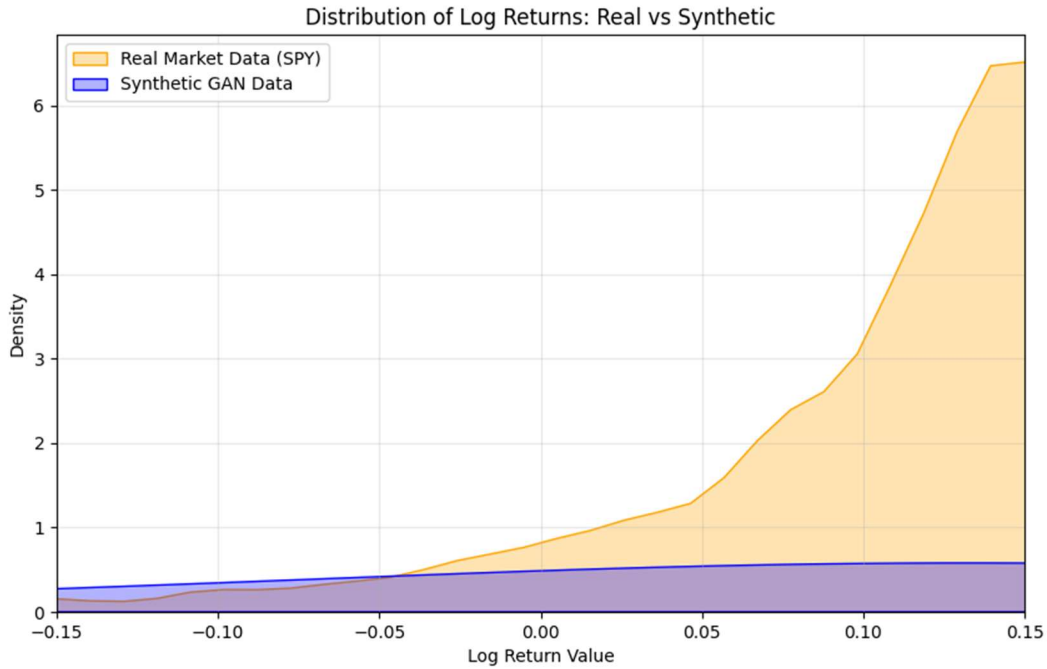


Figure 5: Distribution of Log Returns. The synthetic data (Blue) approximates the real data (Orange) distribution.

## 5. CONCLUSION

This project successfully demonstrated the application of Deep Learning to financial market simulation. By implementing an LSTM-GAN with rigorous architectural stabilization techniques (Batch Normalization and Dropout), we overcame the challenges of non-stationarity and mode collapse.

### Key Findings:

1. **Stationarity is Key:** transforming prices to Log Returns was critical for model convergence.
2. **Architecture Matters:** Standard LSTMs fail in a GAN setting without Batch Normalization to prevent gradient saturation.
3. **Data Utility:** The generated synthetic data successfully replicates the volatility clustering of the S&P 500, making it a viable tool for stress-testing trading algorithms in privacy-sensitive environments.

- 4. Future Work:** To address the distributional shift observed in the PCA analysis, future research should explore Wasserstein GANs (WGAN-GP), which use a more stable distance metric (Earth Mover's Distance) to improve the overlap between real and synthetic distributions.

## 6. REFERENCES

- Cont, R. (2001). *Empirical properties of asset returns: stylized facts and statistical issues*. Quantitative Finance, 1(2), 223-236.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). *Generative adversarial nets*. *Advances in neural information processing systems*, 27.
- Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. *Neural computation*, 9(8), 1735-1780.
- Tsay, R. S. (2010). *Analysis of financial time series*. John Wiley & Sons.
- Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2020). *Quant GANs: deep generation of financial time series*. *Quantitative Finance*, 20(7), 1119-1137.